Объектно-ориентированное программирование

6 семестр
Лекция 3.

Содержание лекции:

- Классификация паттернов GoF
- Одиночка (Singleton)
- Фабричный метод (Factory Method)
- Декоратор (Decorator)
- Адаптер (Adapter)
- Фасад (Facade)
- Как правильно выбирать паттерн

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Классификация паттернов GoF

GoF (Gang of Four) — «Банда четырёх»

GoF — это прозвище четырёх авторов, которые в 1994 году выпустили книгу, ставшую «Библией» программирования: **«Design Patterns: Elements of Reusable Object-Oriented Software»**.

Состав «Банды»:

Эрих Гамма (Erich Gamma)

Ричард Хелм (Richard Helm)

Ральф Джонсон (Ralph Johnson)

Джон Влиссидес (John Vlissides)

Почему это важно:

Систематизация: До этой книги разработчики решали одни и те же задачи по-разному. Авторы выделили и описали **23 классических паттерна**.

Единый язык: Они ввели термины (такие как *Singleton*, *Strategy*, *Observer*), которые сегодня понимает любой программист в любой точке мира.

Стандарт ООП: Книга доказала, что объектно-ориентированное проектирование — это не просто классы, а гибкие и переиспользуемые схемы взаимодействия между ними.

Паттерн — это проверенное временем решение типичной задачи проектирования. Это «золотой стандарт» того, как связать объекты и классы.

Универсальность: Паттерны не зависят от языка программирования (принципы в C#, Java или Python идентичны).

Общий язык: Позволяют разработчикам понимать друг друга быстрее. Вместо долгого объяснения структуры достаточно сказать название паттерна.

Паттерн должен упрощать, а не усложнять.

Избегайте over-engineering: не используйте паттерны «про запас» или ради самих паттернов.

Группы паттернов

1. Порождающие (Creational)

Задача: Гибкое создание объектов.

Суть: Скрывают логику создания (инстанцирования). Позволяют системе не зависеть от того, как создаются, компонуются и представляются её объекты.

Помогают избавиться от бесконечных new в коде.

2. Структурные (Structural)

Задача: Построение связей между объектами.

Суть: Отвечают за то, как собрать из мелких «кирпичиков» (классов) крупную и эффективную систему.

3. Поведенческие (Behavioral)

Задача: Взаимодействие и алгоритмы.

Суть: Определяют, как объекты общаются между собой и как распределяются обязанности.

Подробно разберем на следующей лекции.

Одиночка (Singleton)

Проблема: В приложении существуют ресурсы, которые должны быть уникальны (например, настройки программы, логгер или соединение с базой данных). Создание нескольких копий таких объектов ведет к конфликтам и избыточному потреблению памяти.

Решение:

Гарантировать, что у класса есть **только один** экземпляр.

Предоставить глобальную точку доступа к этому экземпляру.

Отличие от глобальных переменных:

Ленивая инициализация (Lazy Initialization): Объект не создается при старте программы «на всякий случай». Он рождается только в тот момент, когда к нему впервые обратились. Если объект не понадобился — память не будет занята.

Одиночка (Singleton, Синглтон) - порождающий паттерн, который гарантирует, что для определенного класса будет создан только один объект, а также предоставит к этому объекту точку доступа.

Чтобы реализовать этот паттерн, нужно соблюсти три обязательных условия в коде класса:

Приватный конструктор: Мы «запираем» класс изнутри. Никто не может написать new MyClass() за пределами самого класса.

Статическое поле (instance): В нем хранится тот самый единственный экземпляр.

Публичный статический метод (GetInstance): Единственный способ получить объект.

Логика метода GetInstance:

Если поле instance пустое (null) — создаем объект и сохраняем его.

Если объект уже есть — просто возвращаем его.

```
class OS
{
    // 1. Статическое поле для хранения единственного объекта
    private static OS _instance;

    public string Name { get; private set; }

    // 2. Приватный конструктор — запрет на создание через 'new'
    private OS(string name) => Name = name;

    // 3. Глобальная точка доступа
    public static OS GetInstance(string name)
    {
        if (_instance == null)
        {
            _instance = new OS(name);
        }
        return _instance;
    }
}
```

ВНИМАНИЕ!

«Синглтон часто называют '**антипаттерном**', если его используют **везде подряд**. Он создает **сильную связанность** в коде, поэтому используйте его только тогда, когда вам действительно жизненно важна уникальность объекта в рамках всей программы».

Фабричный метод (Factory Method)

Если мы пишем так:

```
if (type == "wood") house = new WoodHouse();
else if (type == "panel") house = new PanelHouse();
```

Проблема: Каждый раз, когда в проект заходит новый тип дома (кирпичный, стеклянный), нам приходится искать и переписывать **все** куски кода, где создаются дома. Это нарушает главный принцип архитектуры: «Код должен быть закрыт для изменений, но открыт для расширений».

Представьте, что у нас есть сложный алгоритм «Застройки района». Он делает много дел: закладывает фундамент, проводит трубы, красит стены.

```
void BuildNeighborhood(Developer developer)
{
    for (int i = 0; i < 10; i++)
    {
        var house = developer.Create(); // Магия: просто просим
"создай"
        Console.WriteLine($"Установлен объект №{i}");
    }
}
```

```
// 1. Абстрактные основы
abstract class House { }

abstract class Developer
{
    public abstract House Create();
}

// 2. Реализация для дерева
class WoodHouse : House { }

class WoodDeveloper : Developer
{
    public override House Create() => new WoodHouse();
}
```

Вариант А (Эко-поселок):

```
Developer myDev = new WoodDeveloper(); // Решили строить из
дерева
BuildNeighborhood(myDev); // Весь район станет деревянным
```

Вариант Б (Спальный район):

```
Developer myDev = new PanelDeveloper(); // Решили строить
панельки
BuildNeighborhood(myDev); // Тот же самый метод
BuildNeighborhood построит бетонные джунгли
```

Резюме: Когда это НЕ зря?

Мы пишем классы Фабрик, когда:

Типы объектов могут добавляться в будущем. Нам не придется перелопачивать всю программу, мы просто добавим новый класс-застройщик.

Мы хотим изолировать создание. Например, создание дома может требовать сложных настроек, о которых основной код программы знать не должен.

Структурные паттерны. Декоратор (Decorator)

Задача: У нас есть 2 вида основы пиццы и 5 видов добавок. Клиент может выбрать любую комбинацию.

Если использовать наследование: нам придется создать класс для каждой

комбинации: ItalianPizzaWithCheese, ItalianPizzaWithTomatoAndCheese, B ulgarianWithCheese...

Результат: Десятки однотипных классов, которые невозможно поддерживать. Это и есть **«взрыв иерархии»**.

Решение: Паттерн Декоратор (Decorator)

Вместо создания новых классов мы будем динамически «оборачивать» объект в дополнительные слои функциональности.

Декоратор — это структурный паттерн, который позволяет динамически добавлять объекту новые обязанности.

Обертка: Декоратор имеет тот же интерфейс, что и объект, который он оборачивает.

Делегирование: Декоратор получает запрос, выполняет свою часть работы (например, добавляет цену сыра) и передает запрос вложенному объекту.

Визуальная схема:

[Сыр [Томаты [Пицца-основа]]] — внешняя «матрешка» знает о внутренней, но сама остается «пиццей» для всей остальной программы.

```
// 1. Базовый абстрактный компонент
abstract class Pizza
{
    protected Pizza(string name)
    {
        Name = name;
    }

    public string Name { get; protected set; }
    public abstract int GetCost();
}
// 2. Базовый декоратор (хранит ссылку на оборачиваемый объект)
abstract class PizzaDecorator : Pizza
{
    protected Pizza pizza;
    public PizzaDecorator(string n, Pizza p) : base(n) =>
this.pizza = p;
}
```

Каждый декоратор просто добавляет свою стоимость к стоимости «вложенной» пиццы.

```
class TomatoPizza : PizzaDecorator
{
    public TomatoPizza(Pizza p)
                        : base(p.Name + ", с томатами", p) { }

    // Считаем общую стоимость: вложенная пицца + цена томатов
    public override int GetCost() => pizza.GetCost() + 3;
}

class CheesePizza : PizzaDecorator
{
    public CheesePizza(Pizza p)
                          : base(p.Name + ", с сыром", p) { }

    public override int GetCost() => pizza.GetCost() + 5;
}
```

Теперь мы можем собрать любую комбинацию прямо в процессе работы программы:

```
// Создаем основу
Pizza myPizza = new ItalianPizza();

// Оборачиваем в томаты
myPizza = new TomatoPizza(myPizza);

// Оборачиваем сверху в сыр
myPizza = new CheesePizza(myPizza);

Console.WriteLine(
                $"{myPizza.Name} | Цена: {myPizza.GetCost()}");
// Результат: Итальянская пицца, с томатами, с сыром | Цена: 18
```

Преимущества (+):

- **Гибкость:** Можно комбинировать добавки как угодно в реальном времени.
- **Чистота кода:** Мы не создаем сотни классов под каждую задачу.
- **Соблюдение принципа единственной ответственности:** Каждый декоратор отвечает только за свою добавку.

Недостатки (-):

- **Сложность настройки:** Создание объекта превращается в «многослойную» конструкцию.
- **Много мелких объектов:** В памяти создается цепочка объектов вместо одного.

Декоратор — это не замена наследованию, это способ избежать его там, где комбинаций слишком много. Если вам нужно добавить поведение 'на лету' и иметь возможность так же легко его снять — это ваш инструмент».

Адаптер (Adapter)

Проблема: Несовместимость интерфейсов

Вы используете стороннюю библиотеку или старый код (Legacy), который нельзя изменять. Его методы и форматы данных не совпадают с тем, что ожидает ваша новая программа.

Аналогия из жизни:

Переходник для розетки: У вас американская вилка (плоские контакты), а розетка европейская (круглые отверстия). Вы не меняете розетку и не отрезаете вилку — вы используете **адаптер-прослойку**.

Картридер: Ноутбук понимает только USB, а данные на карте памяти SD. Картридер — это адаптер, который делает SD-карту «видимой» для USB-порта.

Паттерн Адаптер — это объект-обертка, который скрывает внутри себя «неудобный» класс и переводит его вызовы на язык, понятный вашей системе.

Алгоритм реализации:

Интерфейс: Определяем интерфейс, с которым хочет работать ваша программа.

Класс-Адаптер: Создаем новый класс, который реализует этот интерфейс.

Композиция: Внутри адаптера создаем поле, в котором хранится объект «неудобного» класса.

Делегирование: Когда программа вызывает метод адаптера, он внутри себя вызывает нужные методы старого класса, при необходимости конвертируя данные.

Ситуация:

Ваше приложение: Современное, работает только с форматом **JSON**.

Старая БД: Выдает данные только в формате **XML**. Менять БД нельзя (слишком дорого/опасно).

```
// То, что хочет наше приложение
interface IModernDataConnector
{
    string GetJsonData();
}

// Тот самый "неудобный" старый класс
class LegacyXmlDatabase
{
    public string GetSpecificXml() => "<user>Иван</user>";
}
```

```
// Адаптер - наш "переходник"
class XmlToJsonAdapter : IModernDataConnector
{
    private LegacyXmlDatabase _db = new LegacyXmlDatabase();

    public string GetJsonData()
    {
        string xml = _db.GetSpecificXml();
        // Логика конвертации на лету: XML -> JSON
        return "{ 'user': 'Иван' }";
    }
}
```

Преимущества (+):

- **Инкапсуляция:** Логика преобразования данных скрыта в одном месте (Адаптере).
- **Чистота:** Основной код программы остается чистым и работает только с современными интерфейсами.
- **Гибкость:** Можно заменить старую библиотеку на новую, просто переписав адаптер, не трогая остальное приложение.

Недостатки (-):

- **Усложнение:** Введение дополнительных классов. Если адаптеров становится слишком много, архитектура становится перегруженной.

Фасад (Facade)

Проблема: Избыточная сложность

Ваша система состоит из множества сложных подсистем (логика склада, оплата, почта, база данных). Чтобы выполнить одну бизнес-задачу (например, «Оформить заказ»), программисту приходится вызывать десятки методов в строгом порядке.

Это порождает ошибки и делает код «хрупким».

Аналогия из жизни:

Кнопка «Пуск» в машине: Вам не нужно знать, как работает бензонасос, впрыск топлива и стартер. Вы жмете одну кнопку — машина заводится.

Пульт от ТВ: Вы жмете «Канал 5», не задумываясь о частотах, дешифровке сигнала и работе матрицы.

Фасад — это высокоуровневый класс-прослойка, который предоставляет один простой интерфейс к сложной системе классов.

Как это работает:

Создаем класс-Фасад.

Агрегируем подсистемы: Внутри Фасада создаются экземпляры всех сложных классов.

Упрощаем методы: Пишем 2–3 публичных метода (например, StartProject), внутри которых Фасад сам вызывает методы подсистем в нужном порядке.

Результат: Пользователь вашего кода (клиент) взаимодействует только с Фасадом, не видя «кишок» системы.

```
// Сложные "кишки" системы
class Warehouse
{
    public bool Check() => true;
}

class Payment
{
    public void Pay() => Console.WriteLine("Оплачено");
}

class SmsService
{
    public void Send() => Console.WriteLine("SMS отправлено");
}
```

```
class OrderFacade // Наш ФАСАД
{
    private Warehouse _w = new();
    private Payment _p = new();
    private SmsService _s = new();

    public void PlaceOrder()
    {
        if (_w.Check())
        {
            _p.Pay();
            _s.Send();
            Console.WriteLine("Готово!");
        }
    }
}

OrderFacade market = new OrderFacade();
market.PlaceOrder(); // Всего одна строка вместо сложной логики!
```

Преимущества (+):

Изоляция: Клиенты не зависят от сложности подсистем. Если вы замените класс SmsService на EmailService, клиент этого даже не заметит.

Снижение порога входа: Новому разработчику проще начать работать с вашим кодом через Фасад.

Принцип KISS: Мы делаем использование системы максимально простым.

Недостатки (-):

Риск «Божественного объекта»: Есть соблазн превратить Фасад в гигантский класс, который делает вообще всё в программе.

Как правильно выбирать паттерн

Паттерн — это не цель, а инструмент. Чтобы не превратить код в «архитектурный лабиринт», следуйте простому алгоритму:

Абстрагируйтесь от деталей: Опишите вашу задачу без привязки к предметной области. Вместо «как добавить сыр к пицце» думайте «как динамически расширить функционал объекта».

Определите тип проблемы:

Сложно **создавать** объекты? — Смотрите *Порождающие* паттерны.

Сложно **связать** компоненты в единую структуру? — Вам нужны *Структурные*.

Запутались во **взаимодействии** и алгоритмах? — Это *Поведенческие*.

Изучите «соседей»: Не хватайтесь за первый похожий паттерн. Сравните его с родственными (например, *Декоратор* часто путают с *Адаптером* или *Прокси*).

Оцените последствия: Поймите, какую цену вы заплатите за гибкость (усложнение кода, лишние классы, расход памяти).

⚠ Золотое правило разработчика

Если вы не понимаете до конца, как работает паттерн или какую конкретную проблему он решает в вашем проекте — не используйте его.